Eulalia VoterID System — Final Project Report

1. Executive summary

Eulalia VoterID is a proof-of-concept system for digital voter identity and political affiliation. It combines a .NET 8/PostgreSQL backend, a React/Vite web client, a Flutter mobile application, and Identus/Atala PRISM SSI infrastructure. The final validation shows an integrated PoC with evidence for selected citizen, authentication, SSI, and affiliation-related flows. The project remains a PoC: the available evidence does not establish production readiness, electoral certification, or public production deployment.

2. Project description

The project investigates a digital identity model in which a citizen can register, authenticate, interact with SSI services, and use credential state as part of controlled political affiliation workflows. The architecture keeps application responsibilities in the backend and clients while using Identus/Atala PRISM for the tested SSI interactions.

3. Objectives

- Build a functional PoC for digital voter identity and political affiliation.
- Expose backend services for citizens, authentication, political affiliation, biometrics, SSI, and organization requests.
- Deliver web and mobile interfaces for the principal interaction paths.
- Validate the SSI sequence from invitation through webhook, status, and credential issuance.
- Create traceable evidence that allows reviewers to distinguish working flows from open limitations.

4. Methodology and activities

Development and validation were iterative. The team implemented the application layers, configured the SSI environment, exercised representative flows, and consolidated results in repository evidence. Validation included backend tests, client tests, API integration checks, SSI end-to-end checks, and manual infrastructure verification.

The SSI activity sequence was:

1. Generate an invitation.
2. Receive and process a webhook event.
3. Consult the identity status.
4. Issue and validate a credential.
5. Apply the credential and biometric-related gates to the affiliation operation.

5. Implementation and architecture

Component	Technology and responsibility
Backend	.NET 8 and PostgreSQL; domain endpoints and validation gates
Web	React/Vite client
Mobile	Flutter application
SSI	Identus/Atala PRISM Cloud Agent and Docker services
Documentation/evidence	`eulalia-documents`, including `integration-validation`

The tested Identus environment included APISIX, Cloud Agent, PRISM node, PostgreSQL, Vault, and Swagger UI. The endpoint `/cloud-agent/_system/health` returned version `1.39.0`. This is environment evidence only and should not be read as proof of a publicly operated service.

6. Results and deliverables

The project produced:

- Four public repositories covering backend, frontend, mobile, and Atala PRISM/Identus integration.
- A documentation repository with integration-validation evidence.
- A documented SSI flow covering invitation, webhook, status, and credential issuance.
- Tested application paths for registration, authentication, and related validation gates.
- A consolidated test report with explicit failures, conditional validation, and coverage limitations.

The result is a technically integrated PoC, not a finished electoral product.

7. Testing and validation

Test area	Reported result
Backend	132 tests: 130 passed and 2 failed
Frontend	3/3 passed
Flutter	1/1 passed
API integration	6 tests: 5 passed and 1 conditional validation
SSI end to end	5/5 passed
Consolidated total	147 tests: 144 passed and 3 failed/conditional, reported as 98.0%

Backend coverage was 12.4% for lines and 25% for branches. The two backend failures and the conditional API validation are not hidden by the aggregate figure. The API affiliation condition reflects the requirement that the relevant SSI credential state be established before the operation is accepted. The available client test sets are small and do not constitute comprehensive coverage.

8. Challenges and solutions

- **Multi-component integration:** The system was separated into backend, client, and SSI layers, allowing focused evidence for each part.
- **Credential-dependent affiliation:** Validation gates were retained so that affiliation does not bypass the required SSI and biometric-related conditions.
- **Uneven test maturity:** Results and coverage were reported by component instead of presenting a single unqualified readiness statement.
- **Infrastructure verification:** The Cloud Agent health response and Docker stack evidence provide a reproducible reference for the tested environment.

Platform transition and milestone constraints

The initial project baseline referenced Atala PRISM. During execution, the SSI platform baseline changed because the original Atala PRISM path was no longer available or maintained for this project, requiring integration and validation with Identus. This required adaptation of the integration work, evidence, terminology, and validation scope; therefore, some deliverables should be interpreted against the revised baseline rather than the initial plan.

The journal article/publication deliverable was affected by this transition and by other pre-established milestone conditions. This is documented as a delivery constraint affecting the planned commitments, not as a failure or direct rejection. The final evidence demonstrates the revised Identus-based PoC and does not claim full equivalence with every original Atala PRISM commitment.

9. Lessons learned

1. SSI state must be treated as a business prerequisite, not only as a transport integration.
2. A PoC report is stronger when it records unsuccessful and conditional outcomes alongside successful flows.
3. Component separation helps reviewers understand what was actually validated.
4. Small test suites can prove a path but cannot establish broad reliability, security, scalability, or electoral suitability.
5. A future pilot requires institutional, privacy, security, operational, and compliance work beyond the current code integration.

10. Recommendations and next steps

- Fix the two backend test failures and add regression tests for current validation behavior.
- Increase backend line and branch coverage and expand web and mobile tests.
- Add biometric, service, widget, API, performance, security, and end-to-end test scenarios.
- Define operational controls for credential lifecycle, recovery, revocation, privacy, logging, and data retention.
- Perform an independent security and privacy assessment before any external pilot.
- Define the regulatory and electoral requirements that would apply to a real deployment.
- Keep future demonstrations clearly labelled as PoC or pilot environments until those controls are completed.

11. Conclusion

Eulalia VoterID achieved its PoC purpose: it assembled and exercised a digital identity and political affiliation architecture across backend, web, mobile, and SSI components. The evidence supports selected integrated flows, especially the documented SSI sequence, while also showing meaningful limitations in test failures, conditional validation, coverage, and deployment scope. The project provides a foundation for further research and controlled validation; it does not by itself establish production readiness.

12. Relevant links

- eulalia-backend (https://github.com/democracyonchain/eulalia-backend)
- eulalia-frontend (https://github.com/democracyonchain/eulalia-frontend)
- eulalia-app (https://github.com/democracyonchain/eulalia-app)
- eulalia-atala-identus (https://github.com/democracyonchain/eulalia-atala-identus)
- eulalia-documents (https://github.com/democracyonchain/eulalia-documents)
- Integration-validation evidence (https://github.com/democracyonchain/eulalia-documents/tree/main/integration-validation)
- Integration test reports (https://github.com/democracyonchain/eulalia-documents/tree/main/tests/TestIntegration)

Milestone scope note: The closing video and magazine article are separate milestone deliverables and are not covered by this report. Publication was also affected by the platform transition and other pre-established milestone conditions; this report therefore keeps the publication deliverable out of scope while documenting the constraint.